ENGINEERING / QUICK REFERENCE

FreeRTOS Kernel API

Cortex-M task/ISR quick reference for RTOS-based firmware

TASKS: task.h

`xTaskCreate(fn,name,depth,arg,prio,&h)`
Dynamic task creation; stack depth in StackType_t words.

`xTaskCreateStatic(fn,name,depth,arg,prio,stack,&tcb)`
Task with caller-owned stack and TCB.

`vTaskStartScheduler()`
Start scheduler; generally never returns on success.

`vTaskDelete(h)`
Delete task (NULL = current task).

`vTaskPrioritySet(h,p)`
Set a task priority.

`uxTaskPriorityGet(h)`
Get priority.

`vTaskSuspend(h) / vTaskResume(h)`
Suspend/resume. Not a synchronization primitive.

`taskYIELD()`
Offer CPU to appropriate Ready task.

`xTaskGetCurrentTaskHandle()`
Handle of running task.

`uxTaskGetStackHighWaterMark(h)`
Minimum historically unused stack in words.

TIME AND PERIODIC WORK

`vTaskDelay(pdMS_TO_TICKS(ms))`
Delay relative to now at tick resolution.

`xTaskDelayUntil(&last,period)`
Periodic absolute release; returns if delayed.

`xTaskGetTickCount()`
Tick count from task.

`xTaskGetTickCountFromISR()`
Tick count from ISR (port constraints apply).

`pdMS_TO_TICKS(ms)`
Convert milliseconds to ticks; watch rounding/overflow.

`portMAX_DELAY`
Indefinite wait only with supported configuration.

`pdTRUE / pdFALSE`
Boolean-like values; check each API contract.

QUEUES: queue.h

`xQueueCreate(n,sizeof(Item))`
Create queue of copied objects.

`xQueueCreateStatic(n,size,storage,&cb)`
Static queue; storage >= n * size bytes.

`xQueueSend(q,&item,to)`
Copy item to queue tail (may wait).

`xQueueSendToFront(q,&item,to)`
Copy item to head.

`xQueueReceive(q,&out,to)`
Receive and remove item.

`xQueuePeek(q,&out,to)`
Read without removing.

`uxQueueMessagesWaiting(q)`
Number of queued items.

`xQueueOverwrite(q,&item)`
Overwrite - ONLY on length-1 queue.

`xQueueSendFromISR(q,&item,&woken)`
ISR-safe, never blocks.

`xQueueReceiveFromISR(q,&out,&woken)`
ISR-safe, never blocks.

SEMAPHORES AND MUTEXES: semphr.h

`xSemaphoreCreateBinary()`
Binary semaphore, created empty.

`xSemaphoreCreateBinaryStatic(&cb)`
Static binary semaphore.

`xSemaphoreCreateCounting(max,init)`
Counting semaphore.

FreeRTOS Kernel API

Cortex-M task/ISR quick reference for RTOS-based firmware

`xSemaphoreCreateMutex()`
Task mutex with priority inheritance.

`xSemaphoreCreateMutexStatic(&cb)`
Statically allocated mutex.

`xSemaphoreTake(s,to)`
Take semaphore/mutex in task.

`xSemaphoreGive(s)`
Give semaphore/mutex in task.

`xSemaphoreGiveFromISR(s,&woken)`
FromISR for semaphore, NOT mutex.

`xSemaphoreTakeFromISR(s,&woken)`
FromISR only for binary/counting semaphore.

`xSemaphoreCreateRecursiveMutex()`
Use recursive take/give APIs.

DIRECT NOTIFICATIONS: task.h

`xTaskNotifyGive(task)`
Increment recipient notification count.

`vTaskNotifyGiveFromISR(task,&woken)`
ISR increment notification count.

`ulTaskNotifyTake(pdTRUE,to)`
Wait; clear count to zero.

`ulTaskNotifyTake(pdFALSE,to)`
Wait; decrement count by one.

`xTaskNotify(task,bits,eSetBits)`
OR event bits into notification value.

`xTaskNotifyWait(clearIn,clearOut,&value,to)`
Wait for notification state; read value.

`xTaskNotifyFromISR(task,v,action,&woken)`
ISR notification action.

`xTaskNotifyGiveIndexed(task,idx)`
Increment indexed notification.

`ulTaskNotifyTakeIndexed(idx,clear,to)`
Wait on indexed slot.

EVENT GROUPS: event_groups.h

`xEventGroupCreateStatic(&cb)`
Create event bit group statically.

`xEventGroupSetBits(g,mask)`
Set bits from task context.

`xEventGroupWaitBits(g,bits,clear,all,to)`
Wait for any or all selected bits.

`xEventGroupClearBits(g,mask)`
Clear selected bits.

`xEventGroupSetBitsFromISR(g,bits,&woken)`
Defer set operation to daemon; check result.

`xEventGroupGetBits(g)`
Read group bits from task.

TIMERS AND BYTE BUFFERS

`xTimerCreate(...,period,reload,id,cb)`
Create SW timer: callback in daemon task.

`xTimerCreateStatic(...,&cb)`
Statically allocated software timer.

`xTimerStart(t,to)`
Queue start request to timer daemon.

`xTimerChangePeriod(t,period,to)`
Change timer period.

`xTimerStartFromISR(t,&woken)`
Start request from interrupt.

`xStreamBufferCreate(size,trigger)`
Byte stream buffer.

`xStreamBufferSend(sb,data,n,to)`
Send bytes.

`xStreamBufferReceive(sb,out,n,to)`
Receive variable number of bytes.

`xMessageBufferCreate(bytes)`
Discrete variable-length messages.

FreeRTOS Kernel API

Cortex-M task/ISR quick reference for RTOS-based firmware

`xMessageBufferSend(mb,ptr,n,to)`
Send one complete message.

ISR AND CORTEX-M PRIORITIES

`BaseType_t woken = pdFALSE;`
Initialize context-switch flag.

`vTaskNotifyGiveFromISR(task,&woken)`
Signal a task from interrupt.

`portYIELD_FROM_ISR(woken)`
Request task switch on ISR exit.

`configMAX_SYSCALL_INTERRUPT_PRIORITY`
Cortex-M encoded interrupt-priority ceiling.

`configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY`
Unshifted NVIC priority threshold (if defined).

`taskENTER_CRITICAL() / taskEXIT_CRITICAL()`
Short task-context critical section.

`configTICK_RATE_HZ`
RTOS tick, not a high-resolution PWM clock.

STATIC ALLOCATION, ERRORS, CONFIG

`configSUPPORT_STATIC_ALLOCATION=1`
Enable static API objects.

`configSUPPORT_DYNAMIC_ALLOCATION=0`
Disable heap allocation when supported.

`configCHECK_FOR_STACK_OVERFLOW`
Enable overflow detection.

`configASSERT(expr)`
Detect violated invariants.

`vApplicationStackOverflowHook(...)`
Application stack error hook.

`vApplicationMallocFailedHook()`
Allocation failure hook (dynamic builds).

`configUSE_TIMERS=1`
Enable timer service.

`configTASK_NOTIFICATION_ARRAY_ENTRIES`
How many notification indexes each task has.

`INCLUDE_xTaskDelayUntil=1`
Enable xTaskDelayUntil API.

KEY CONSIDERATIONS

SAFETY: ISR must never block or use a mutex. Cortex-M numerically

LOW NVIC value means HIGH urgency: with

configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY=5, ISR priorities 0..4

may NOT call FreeRTOS FromISR APIs. Queues copy items; software

timer callbacks must not block.

OFFICIAL DOCUMENTATION

`https://www.freertos.org/Documentation/`

`https://github.com/FreeRTOS/FreeRTOS-Kernel`